

# Artificial Intelligence

Course Code: CS-202

Topic: Uninformed Search Algorithms

Instructor: Habiba Arshad

# Week-5

This week lecture will cover the following CLO:

- **CLO 2:** . Implement artificial intelligence techniques and case studies

# Uniform Cost Search

- Uniform Cost Search (UCS) is a popular search algorithm used in artificial intelligence (AI) for finding the least cost path in a graph.
- It is a variant of Dijkstra's algorithm and is particularly useful when all edges of the graph have different weights, and the goal is to find the path with the minimum total cost from a start node to a goal node.

# Uniform Cost Algorithm

1. Insert the root into the queue
2. While the queue is not empty
  - Dequeue the maximum priority (minimum cost) element from the queue
  - (If priorities are same, alphabetically smaller path is chosen)
  - If the path is ending in the goal state, print the path and exit
  - Else
  - Insert all the children of the dequeued element, with the cumulative costs as priority

**Note:** Example solved in class

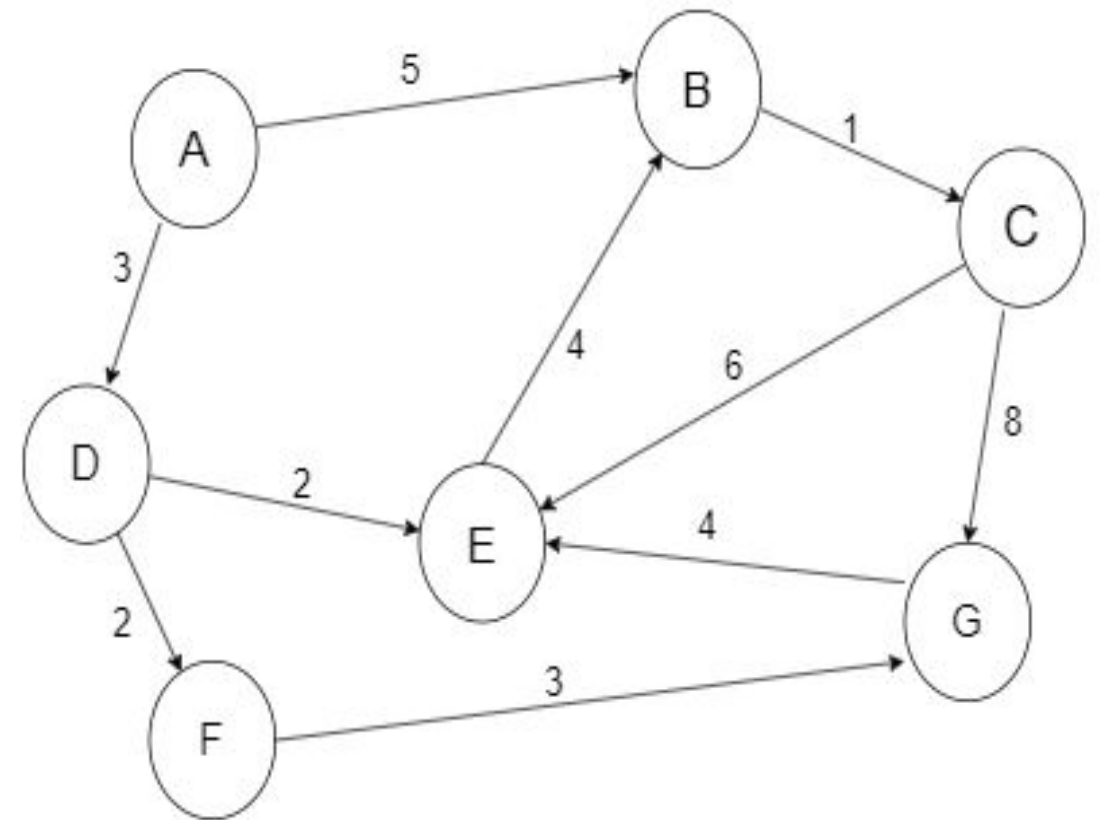
---

# Problem Statement

- Imagine you are in a city and want to travel from your home (Node A) to a friend's house (Node G). The city map is represented as a graph where nodes are locations and edges are roads with associated travel costs (e.g., distance, time, or fuel).

# Uniform Cost Example

|    | Frontier List                                          | Expand List | Explored List                          |
|----|--------------------------------------------------------|-------------|----------------------------------------|
| 1. | {{A,0}}                                                | A           | NULL                                   |
| 2. | {{A-D, 3}, {A-B, 5}}                                   | D           | {A}                                    |
| 3. | {{A-B, 5}, {A-D-E, 5}, {A-D-F, 5}}                     | B           | {A, D}                                 |
| 4. | {{A-D-E, 5}, {A-D-F, 5}, {A-B-C, 6}}                   | E           | {A, D, B}                              |
| 5. | {{A-D-F, 5}, {A-B-C, 6}, <del>{A-D-E-B, 9}</del> }     | F           | {A, D, B, E}                           |
|    | *here B is <b>already explored</b>                     |             |                                        |
| 6. | {{A-B-C, 6}, {A-D-F-G,8}}                              | C           | {A, D, B, E, F}                        |
| 7. | {{A-D-F-G,8}, <del>{A-B-C-E,12}</del> , {A-B-C-G, 14}} | G           | {A, D, B, E, F, C}                     |
|    | *here E is <b>already explored</b>                     |             |                                        |
| 8. | {{A-D-F-G,8}}                                          | NULL        | {A, D, B, E, F, C, G}<br># GOAL Found! |



| Step | Expanded Node         | Queue Before Expansion (Cost, Node) | Neighbors Added / Updated                        | Queue After Expansion (Cost, Node) |
|------|-----------------------|-------------------------------------|--------------------------------------------------|------------------------------------|
| 1    | Start at <b>A (0)</b> | []                                  | Expand A → B (5), D (3)                          | [(3, D), (5, B)]                   |
| 2    | <b>D (3)</b>          | [(3, D), (5, B)]                    | Expand D → E (5), F (5)                          | [(5, B), (5, E), (5, F)]           |
| 3    | <b>B (5)</b>          | [(5, B), (5, E), (5, F)]            | Expand B → C (6), E (9)<br>(but E already has 5) | [(5, E), (5, F), (6, C)]           |
| 4    | <b>E (5)</b>          | [(5, E), (5, F), (6, C)]            | Expand E → C (11), G (9)                         | [(5, F), (6, C), (9, G)]           |
| 5    | <b>F (5)</b>          | [(5, F), (6, C), (9, G)]            | Expand F → G (8) →<br>update G (9 → 8)           | [(6, C), (8, G)]                   |
| 6    | <b>C (6)</b>          | [(6, C), (8, G)]                    | Expand C → G (14), but<br>G already has 8        | [(8, G)]                           |
| 7    | <b>G (8)</b>          | [(8, G)]                            | Goal reached                                     | Finished                           |

# Uniform Cost Search

## Advantages

- Guarantees optimality and completeness.
- Works with weighted graphs.

## Disadvantages

- High time and space complexity.
- Inapplicable to negative edge costs.
- No heuristic guidance, making it less efficient.
- Not suitable for real-time or time-sensitive applications.



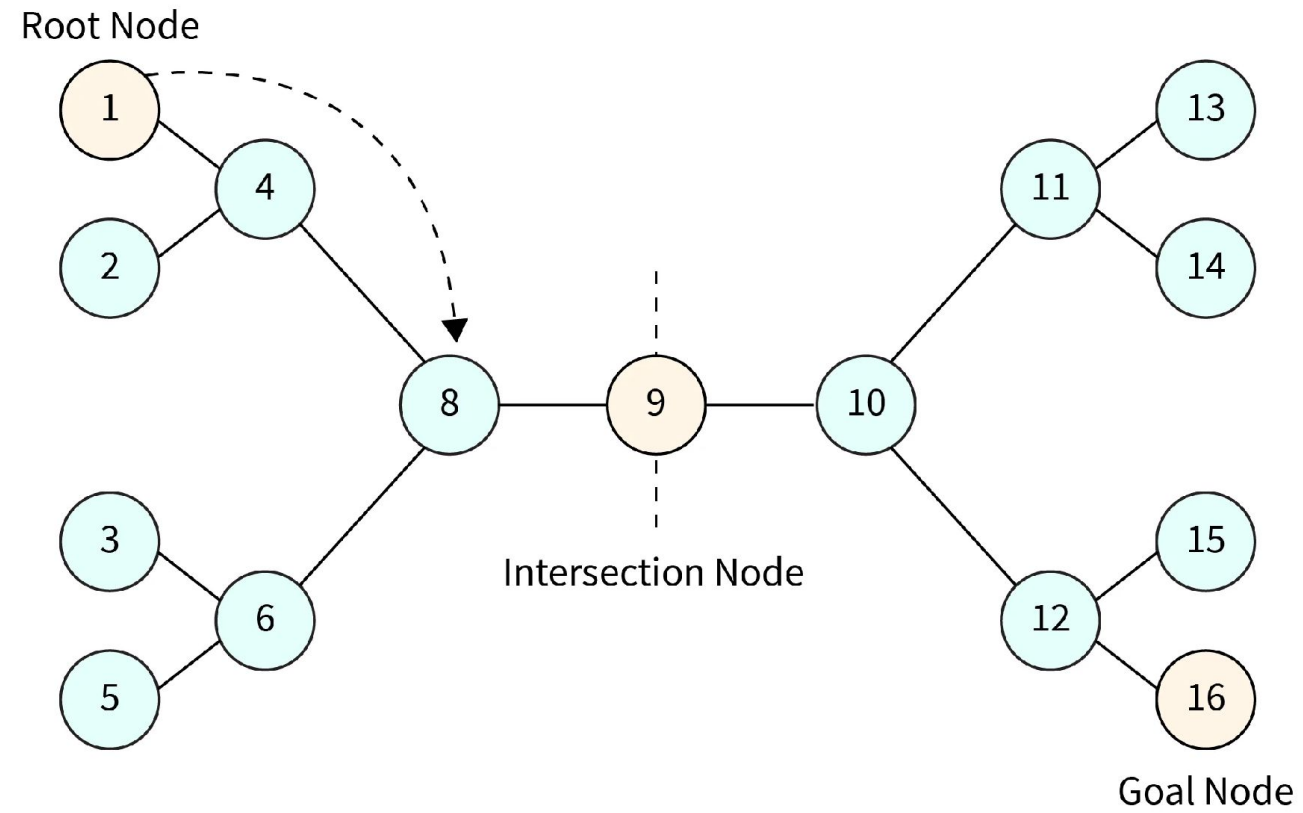
# Bidirectional Search

- Bidirectional Search is Graph Search Algorithm where two graph traversals (BFS) take place at the same time and is used to find the shortest distance between a fixed start vertex and end vertex.
- It significantly reduces the amount of exploration done. It is implemented using the Breadth First Search (BFS) Algorithm.
- BFS is run simultaneously on two vertices - the start and the end vertex. One single BFS tree is now replaced by two sub trees, and the search is terminated when the two trees intersect.

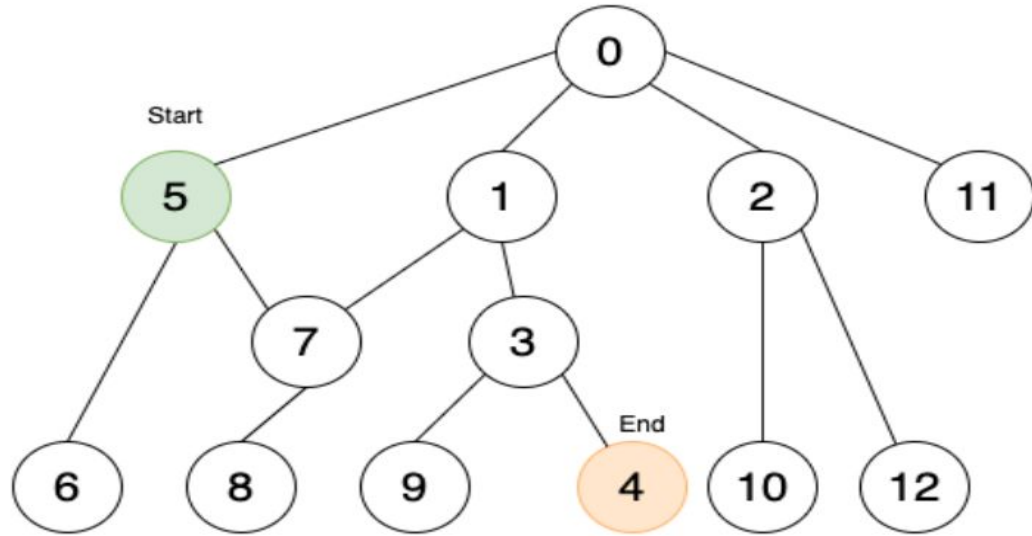
# Bidirectional Search Algorithm

- Start(Q) = Queue for BFS from start node
- End(Q) = Queue for BFS from end node
- parent= Array where startparent[i] is parent of node i
- visited= Array where visited[i]=True if node i has been encountered
  - while start(Q) is not empty and end(Q) is not empty
  - perform next iteration of BFS for start(Q) (also save the parent of children in parent array)
  - perform next iteration of BFS for end(Q)
  - if we have encountered the intersection node
    - save the intersection node
    - break
- using intersection node, find the path using parent array

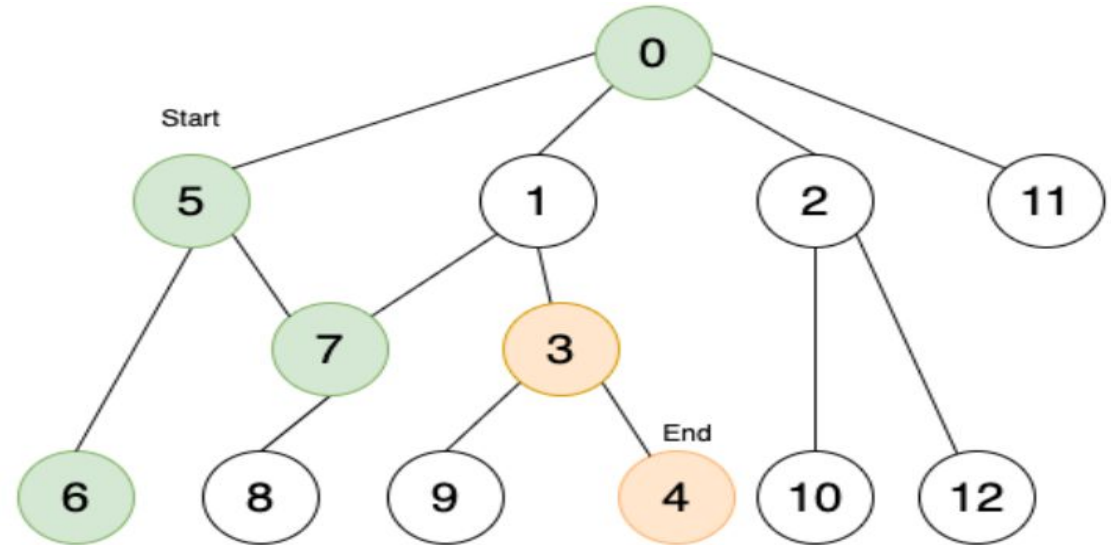
## Bidirectional Search



# Bidirectional Search Example

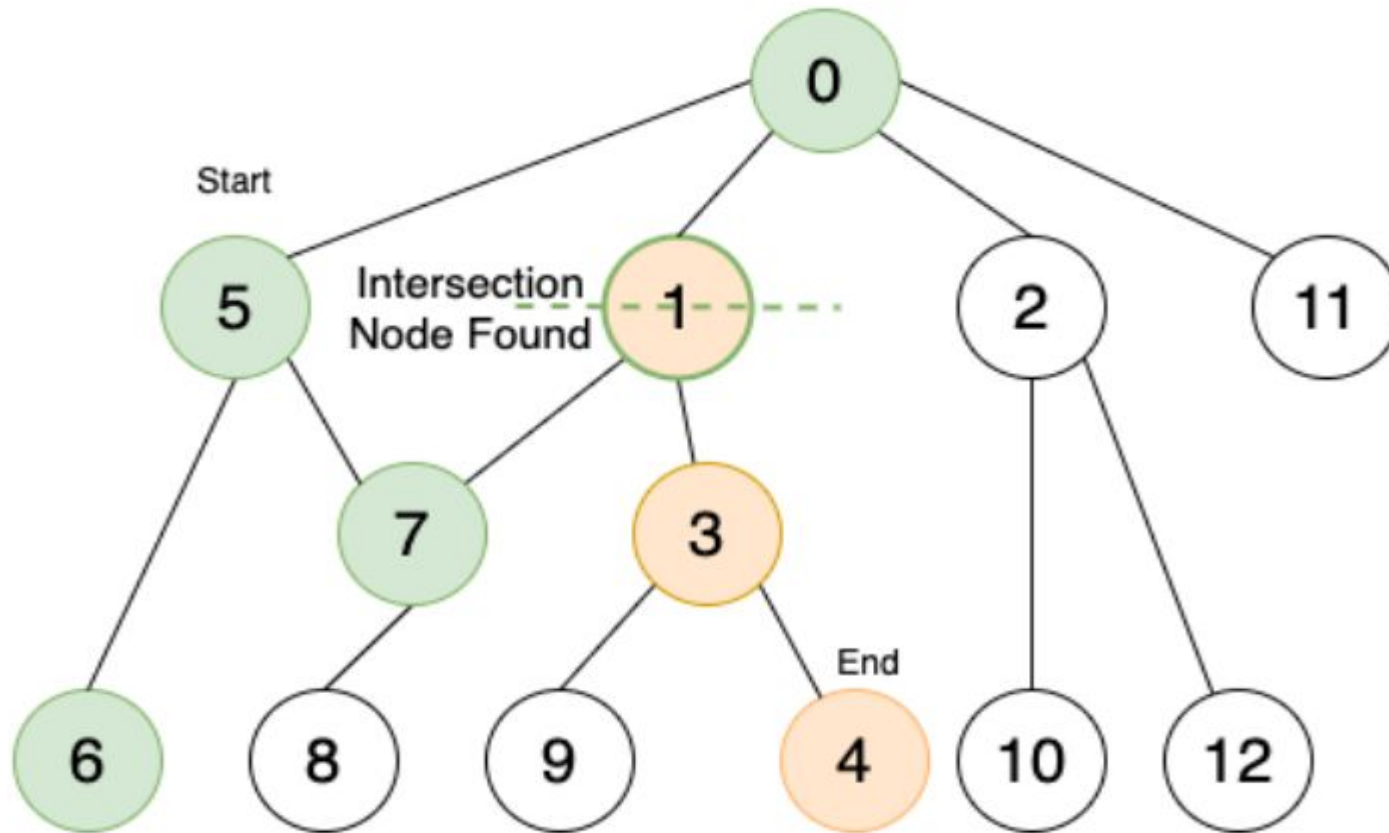


- The start node is 5 and the end node is 4.  
Aim: To find the shortest path from 5 to 4 using bidirectional search.
- Do BFS from both directions.
- Start moving forward from start node (Green) and backwards from end node (Orange).



- Similar to BFS, at every point explore the next level of nodes till you find an intersecting node.

# Bidirectional Search Example



- Stop on finding the intersecting node.
- Trace back to find the path
- Path: 5 – 0 – 1 – 3 – 4

# Bidirectional Search

## Advantages

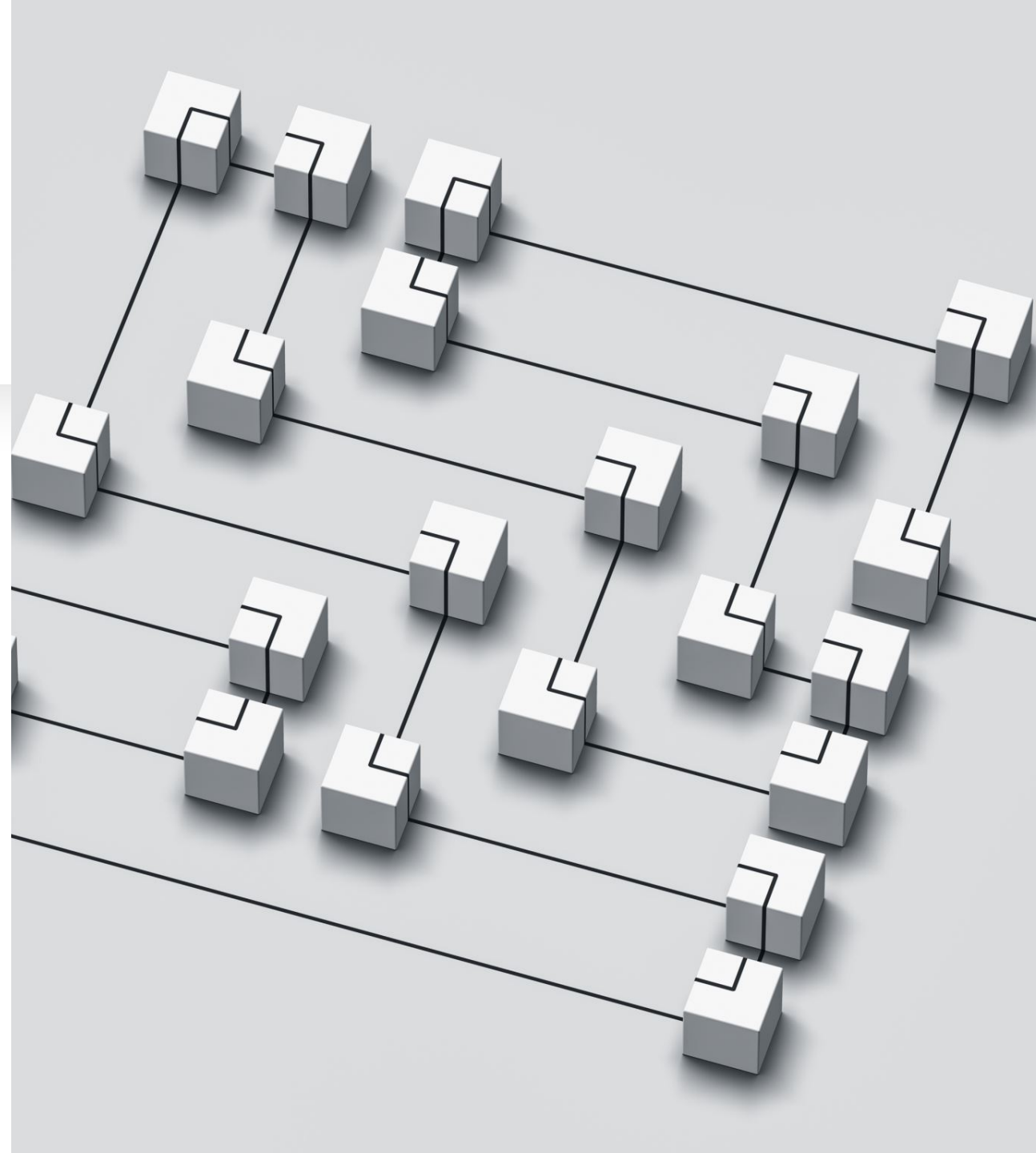
- Reduced Search Space
- Lower Time Complexity  $O(b^{d/2})$
- Memory Efficiency
- Optimal Solution

## Disadvantages

- Requires Goal State to Be Known
- Complex Implementation
- Not Suitable for All Problem Types

# Importance of Search in AI

- Provides a systematic way to explore complex problem spaces.
- Enables AI to solve a wide variety of problems (e.g., pathfinding, scheduling, game playing).
- Forms the backbone of more advanced techniques like machine learning and planning.



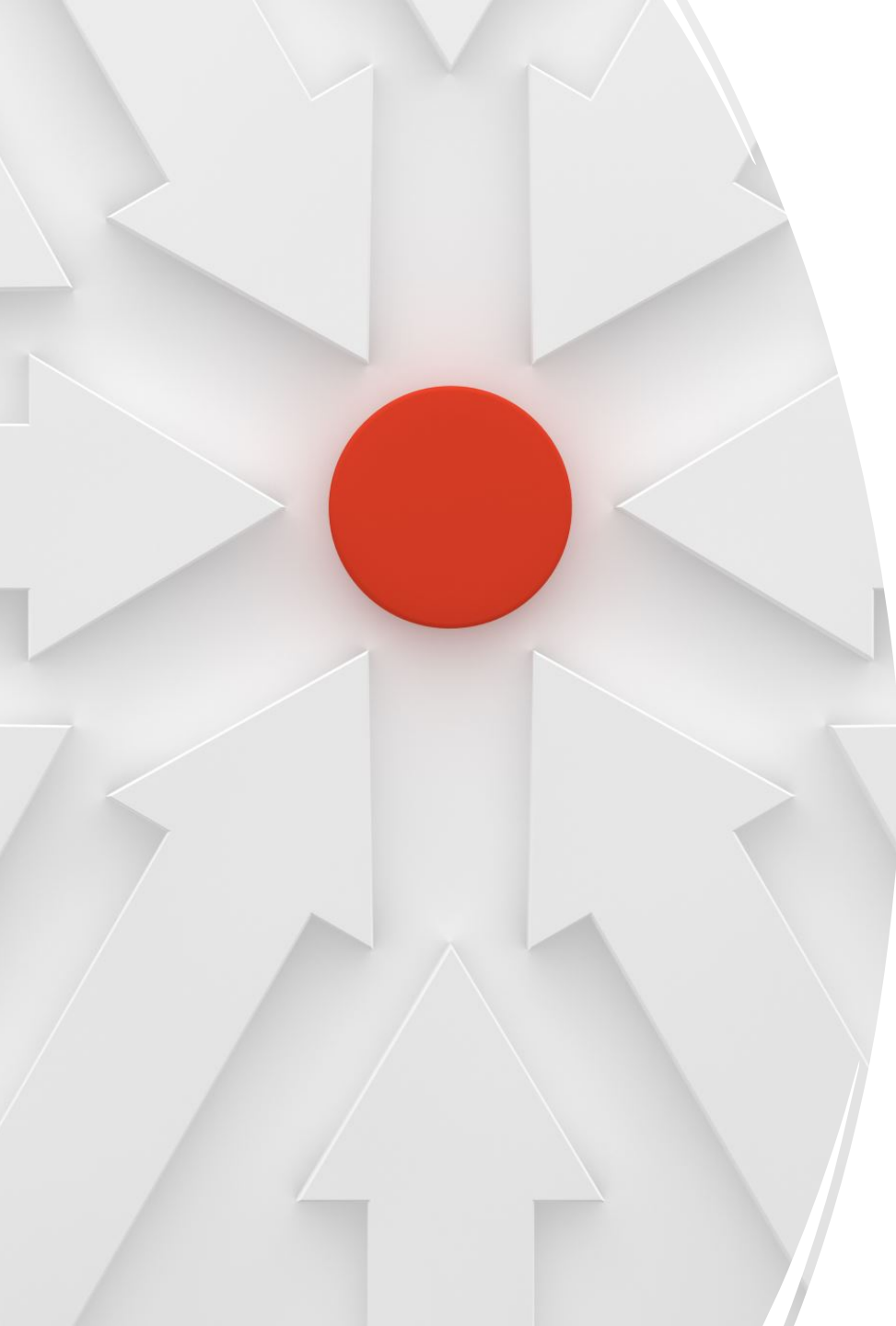


# Common Searching Algorithms

---

- **Uninformed Algorithms:** BFS, DFS, Uniform Cost Search.
- **Informed Algorithms:** Greedy Best-First, A\* Search.
- **Other Techniques:**
  - Iterative Deepening (combines depth-limited search with DFS).
  - Bidirectional Search (searches from both start and goal).





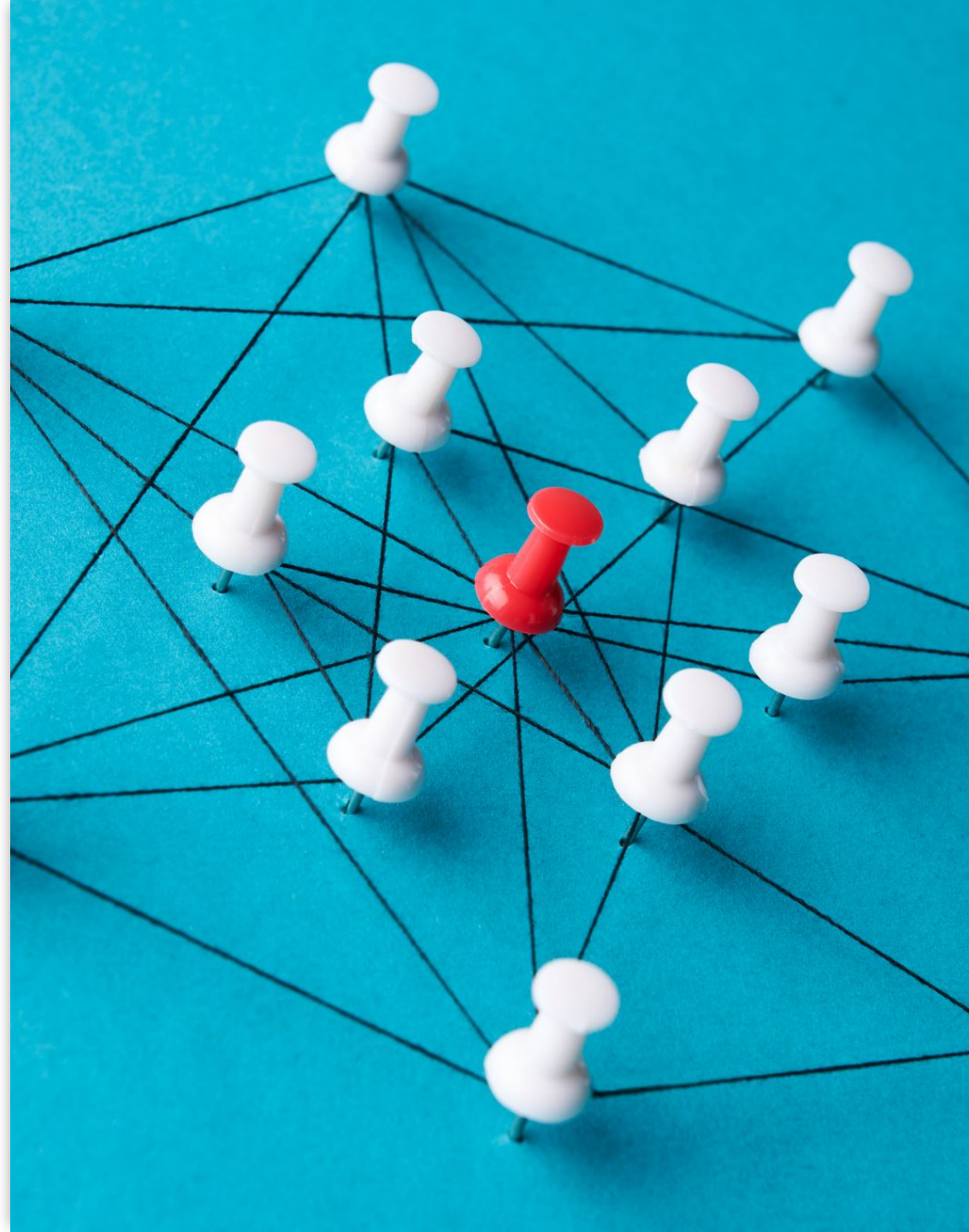
# Heuristics in Informed Search

---

- A heuristic is a problem-specific estimate that helps guide the search towards the goal.
- Good heuristics can significantly reduce search time.
- Examples: Manhattan distance in grid-based pathfinding, straight-line distance in navigation.

# Comparison of Search Techniques

- Efficiency, optimality, and completeness of each algorithm.
- When to use uninformed vs. informed search.



# Summary

---

Problem-solving agents use search techniques to find solutions.

---

Uninformed search explores the search space without extra information.

---

Informed search uses heuristics to guide the search process.

---

A\* is a powerful informed search algorithm combining cost and heuristic estimates.